

## المحاضرة الثانية: أساسيات إطار Dart



م. سليمان الشوصي

## ١. التحديد الصريح للنوع (Explicit Typing)

هنا نقوم بتحديد نوع البيانات بدقة قبل اسم المتغير (مثل `int` للأعداد الصحيحة، `double` للأعداد العشرية، `bool` للقيم المنطقية، و `String` للنصوص).

ميزتها: تجعل الكود مقروءاً جداً ومحمياً من الأخطاء؛ فالبرنامج لن يسمح بتخزين نص داخل متغير من نوع `int` وسيظهر خطأ أثناء الكتابة مباشرة.

## ٢. الاستنتاج التلقائي للنوع (Type Inference باستخدام var)

نستخدم الكلمة المفتاحية `var` دون تحديد النوع يدوياً. هنا تقوم لغة Dart בזكاء باستنتاج نوع المتغير تلقائياً بناءً على القيمة الأولى المسندة إليه (فمثلاً تعتبر `a` كـ `int` وتعتبر `s` كـ `String`).

ميزتها: تعطي مرونة وسرعة في كتابة الكود.

## ٣. الثوابت أثناء وقت التجميع (Compile-time Constants باستخدام const)

نستخدم `const` لتعريف قيمة ثابتة بشكل مطلق لا يمكن تعديلها نهائياً بعد كتابتها. يجب أن تكون قيمة المتغير معروفة وثابتة أثناء كتابة الكود وتجميعه (Compile-time).

ميزتها: تحجز مساحة ثابتة ومحسنة في الذاكرة ولا يمكن للبرنامج تعديلها أثناء التشغيل، مما يحسن الأداء.

## ٤. الثوابت أثناء وقت التشغيل (Runtime Constants باستخدام final)

يشبه الـ `const` في أنه لا يمكن تعديل قيمته بعد إسنادها لأول مرة (تُعطى القيمة لمرة واحدة فقط).

الفرق الجوهرى بين `const` و `final`:

`const` يجب أن تكون قيمته معروفة مسبقاً أثناء كتابة الكود.

`final` يمكن أن يحصل على قيمته أثناء تشغيل التطبيق (Runtime)، مثل جلب تاريخ اليوم الحالي من الجهاز أو جلب بيانات من الإنترنت؛ بمجرد وصول القيمة للمتغير لأول مرة يتم قفلها ولا يمكن تعديلها بعد ذلك.

```
void main() {
  // أنواع طرق تعريف المتغيرات
  // النوع الاول 1
  int x = 5;
  double y = 5.5;
  bool b = true;
  String st = "suliman";
  print("x=${x + 5}"); // "x=$x + 5" // "x=${x + 5}"
  // الطريقة الثانية 2
  var a = 20;
  var p = 20.5;
  var f = false;
  var s = "alshossi";

  a = 30;
  print(a);
  // تعريف المتغيرات الثابتة 3
  const c1 = 10;
  c1=3; // لا يمكن تغيير قيمة لانها من نوع ثابت
  print(c1);
  // تعريف المتغيرات ذات القيم النهائية 4
  final f1 = 15.5;
  f1 = 10;
  // لا يمكن تبديل القيمة لانه لاتعطى القيمة الا مرة واحدة فقط
}
```

```
void main() {
  // انواع المصفوفات
  var a = 20, p = 20.5, b = true, st = "suliman", n,
      h = null, x; p = 10;
  print(a); print(p); print(b);
  print(st); print(n); print(h); print(p);
  // -----1 List or Array
  var d = [1, 2, 3, 4, 5]; //<int>
  print(d);
  var l = [20, 10.5, true, "sulinan"]; // [1, 2, 3, 4, 5];
  print(l);
  // -----2 set-----
  var y = {20, 10.5, true, "sulinan"};
  print(y);
  // -----3 Map or Dictionary
  var z = {"english": 70, "islamic": 80, "arabic": 85};
  //<String,int>
  print(z);}
```

## المخرجات

```
20
10.0
true
suliman
null
null
10.0
[1, 2, 3, 4, 5]
[20, 10.5, true, sulinan]
{20, 10.5, true, sulinan}
{english: 70, islamic: 80, arabic: 85}
```

## أنواع مجموعات البيانات (Collections / Arrays)

في لغة Dart، لا توجد تسمية تقليدية باسم "Array"، بل تنقسم المجموعات إلى ثلاثة أنواع رئيسية لكل منها سلوك واستخدام محدد:

## ١. القوائم (List):

- هي عبارة عن سلسلة مرتبة من العناصر (Ordered Collection)، حيث يحمل كل عنصر رقماً دليلاً يبدأ من الصفر يسمى الـ Index (مثال: [0] قيمته ٢٠).
- ميزتها: تسمح بتكرار القيم وتحافظ على نفس ترتيب إدخال العناصر بشكل دقيق.

## ٢. المجموعات الفريدة (Set):

- تشبه الـ List في فكرة تجميع العناصر، ولكنها تُعرّف باستخدام الأقواس المجددة {} بدلاً من الأقواس المربعة [].
- ميزتها: تمنع تكرار أي عنصر تماماً. إذا حاولت إضافة عنصر موجود مسبقاً، سيتجاهله البرنامج تلقائياً. وهي ممتازة لتصفية البيانات أو عند التأكد من فريدة العناصر (مثل أرقام الهواتف أو المعارف الشخصية).

## ٣. الخرائط أو القواميس (Map):

- هيكل بيانات يخزن العناصر على هيئة ثنائيات: مفتاح وقيمة (Key-Value Pair).
- ميزتها: لا نصل إلى القيمة باستخدام رقم الـ Index، بل نصل إليها عبر الـ Key الخاص بها (مثال: z["english"] يعطي النتيجة ٧٠). ويشترط هنا أن تكون المفاتيح فريدة تماماً لا تتكرر، بينما يمكن للقيم أن تتكرر بشكل طبيعي. وهي تشبه جداً ملفات الـ JSON المستخدمة في تبادل البيانات عبر الإنترنت.

```
void main() {
  // انواع الجمل الشرطية
  // الطريقة الاعتيادية المعروفة 1
  int x = 5;
  int y = 10;
  int largNumper;
  if (x > y) {largNumper = x;}
  else {largNumper = y;}
  print(largNumper);
  // الطريقة 2 عن طريق استخدام ال expation
  largNumper = x > y ? x : y;
  print(largNumper);
}
```

```
void main() {
  // الطريقة الثالثة
  String myName = "Suliman"; //null;
  var name = myName ?? "alshossi";
  print(name); // print("${myName ?? "alshossi"}");
}
```

### ١. الجملة الشرطية التقليدية (if-else)

- هي الطريقة الكلاسيكية والأكثر وضوحاً للقراءة. تفحص شرطاً معيناً؛ فإذا تحقق نفذت الكود الأول، وإذا لم يتحقق انتقلت لتنفيذ كود الـ **else**.
- **ميزتها:** ممتازة للمبتدئين وتستخدم عندما يحتوي الشرط على أسطر برمجية متعددة ومعقدة.

### ٢. المعامل الثلاثي (Ternary Operator) : هي اختصار ذكي يدمج جملة if-else كاملة في سطر واحد فقط.

Condition ? Expression 1 (if True) : Expression 2 (if False)

- **ميزتها:** تختصر الكود وتستخدم بكثرة داخل واجهات Flutter لعرض أو إخفاء عناصر التصميم بناءً على حالة معينة.

### ٣. معامِل التحقق من القيمة الفارغة (Null-Coalescing Operator) :

- هو معامِل مخصص للتعامل مع القيم الفارغة (null). يفحص المتغير الأيسر: إذا كان يحتوي على قيمة فعلية يستخدمها، وإذا كان فارغاً (null) يستخدم القيمة البديلة المكتوبة على اليمين.
- **ميزتها:** تعتبر حائط الصد الأول لحماية التطبيق من الانهيار (Crash) بسبب البيانات المفقودة، وهي جزء أساسي من ميزة Null Safety في لغة Dart.

## الحلقات التكرارية في Dart

## ١. الحلقة التقليدية (Classic for Loop)

- الفكرة: تعتمد على عدّاد رقمي (i) ومؤشر لشرط التوقف وزيادة العداد يدوياً.
- تستخدم: عندما تحتاج لمعرفة موقع العنصر الحالي (Index) أو عندما نريد التحكم في مسار العد (مثل القفز خطوتين، أو البدء من النهاية).

## ٢. حلقة العنصر (For-In Loop):

- هي حلقة تمر تلقائياً على عناصر المصفوفة واحداً تلو الآخر من البداية للنهاية.
- الاستخدام: لقراءة وعرض جميع العناصر بكود أنظف وأكثر أماناً ضد خطأ تجاوز حدود المصفوفة (Index Out of Range)

## ٣. دالة المرور التلقائي (forEach Method):

- هي دالة برمجية جاهزة مدمجة داخل المصفوفة، تأخذ دالة أخرى (Anonymous Function) لتنفيذها على كل عنصر.
- الاستخدام: لتطبيق عمليات برمجية سريعة ومباشرة على العناصر بأسلوب البرمجة الوظيفية (Functional Programming).

```
void main() {
  // انواع الحلقات التكرارية
  var list = [20, 10.5, true, "sulinan"];
  //1
  for (int i = 0; i<list.length.toInt(); i++)
  { print(list[i]);}
  //2
  print("*****");
  for (var i in list) {
    print(i);
  }
  //3
  print("-----");
  list.forEach((i) {
    print(i);
  });}
}
```

```
20
10.5
true
sulinan
*****
20
10.5
true
sulinan
-----
20
10.5
true
sulinan
```

```

void main() {
    // التحكم في الحلقات التكرارية
    //1- طريقة إيقاف التكرار
    for (int i = 1; i <= 12; i++) {
        for (int j = 1; j <= 12; j++) {
            print("$i*$j= ${i * j}");
            if (j == 5) //i==5
            {break;}}
        print("*****");
    //2- طريقة إيقاف التكرار
    sulimna:
    for (int i = 1; i <= 12; i++) {
        alshossi:
        for (int j = 1; j <= 12; j++) {
            print("$i*$j= ${i * j}");
            if (j == 5) { break sulimna; } }}
        print("-----");
    //3- طريقة الاستمرار التكرار
    sulimna:
    for (int i = 1; i <= 12; i++) {
        alshossi:
        for (int j = 1; j <= 12; j++) {
            if (j == 5) { continue sulimna; }
            print("$i*$j= ${i * j}"); }}
    }
}

```

### طرق التحكم في الحلقات التكرارية (Loop Control)

#### ١. الإيقاف الافتراضي (break)

```
if (j == 5) { break; }
```

- يقوم الأمر **break** الافتراضي بإنهاء وإيقاف الحلقة الداخلية القريبة منه مباشرة (حلقة j).
- من خلال الكود: بمجرد أن تصل قيمة j إلى ٥ في جدول الضرب، يتم إنهاء حلقة الضرب الحالية لـ i والذهاب للسطر التالي، ثم تبدأ الدورة التالية للمتغير i (الخارجي) بشكل طبيعي (مثلاً ينتقل من جدول الـ ١ إلى جدول الـ ٢).
- الخلاصة: **break** الافتراضي يكسر الغرفة التي هو فيها فقط (الحلقة الداخلية الحالية).

#### ٢. الإيقاف المُسمى (Labeled break)

```
if (j == 5) { break sulimna; }
```

- نستخدم هنا ميزة التسمية (Labels)؛ حيث قمنا بتسمية الحلقة الخارجية باسم **sulimna**.
- من خلال الكود: بمجرد أن تصل j إلى ٥ في الدورة الأولى (عندما i = 1)، فإن الأمر **break sulimna** يقوم بإنهاء وإغلاق الحلقة الخارجية بأكملها فوراً، وبالتالي يتوقف برنامج الضرب بالكامل ولا يطبع أي جداول أخرى.
- الخلاصة: **break** المسمى يكسر المبنى بالكامل (يخرج من الحلقة الخارجية المحددة بالاسم).

#### ٣. الاستمرار المُسمى (Labeled continue)

```
if (j == 5) { continue sulimna; }
```

- نستخدم التسمية لتخطي (Skip) ما تبقى من الدورة الحالية للذهاب فوراً إلى الدورة التالية للحلقة المحددة.
- من خلال الكود: بما أن جملة الطباعة **print** تأتي بعد الشرط في مثالك المحدث؛ فبمجرد أن تصبح j == 5، يتجاهل البرنامج السطر الحالي لـ **print** وأي تكرارات متبقية في الحلقة الداخلية لـ j (من ٥ إلى ١٢)، ويقفز فوراً لبدء الدورة الجديدة التالية في الحلقة الخارجية **sulimna** (أي ينتقل إلى الرقم التالي في العداد مباشرة لتبدأ الحلقة الداخلية من j = 1 مجدداً).
- الخلاصة: **continue** المسمى: ينهي العمل الحالي في هذه الشقة فوراً وينتقل فوراً لباب الشقة التالية في الطابق الأعلى (الدورة التالية للحلقة الخارجية المحددة).

## أصناف الدوال الأساسية

```
void main() {
  // انواع الدوال
  sum1();
  print(sum2());
  sum3(3, 4);
  print(sum4(3, 4));
}
```

```
void sum1() {print(7 + 5);}
| Ctrl+L to chat, Ctrl+K to generate
int sum2() { return 7 + 5;}

sum3(x, y) { print(7 + 5);}

sum4(x, y) { return 7 + 5;}
```

## الدالة الأولى sum1 (دالة إجرائية صامتة - Procedure)

- دالة لا تستقبل أي مدخلات، ولا تعيد أي قيمة (نوع إرجاعها void).
- وظيفتها: تقوم بتنفيذ عملية داخلية مباشرة (مثل الطباعة) بمجرد استدعائها دون التفاعل مع أجزاء الكود الأخرى.

## الدالة الثانية sum2 (دالة إرجاع بلا مدخلات - Getter/Producer):

- دالة لا تستقبل مدخلات، ولكنها تعيد قيمة من نوع محدد (مثل int) باستخدام الكلمة المفتاحية return.
- وظيفتها: توليد قيمة أو إرسال بيانات ثابتة ومحددة إلى مكان الاستدعاء لتستخدم في عمليات أخرى.

## الدالة الثالثة sum3 (دالة استقبال بلا إرجاع - Consumer)

- دالة تستقبل مدخلات (Parameters) لتعمل عليها، ولكنها لا تعيد أي قيمة للمستدعي (دالة ديناميكية الإرجاع افتراضياً).
- وظيفتها: استقبال البيانات الخارجية ومعالجتها أو طباعتها فوراً داخل جسم الدالة دون رد ناتج.

## الدالة الرابعة sum4 (الدالة الكاملة - Standard Function)

- الدالة البرمجية المتكاملة؛ حيث تستقبل مدخلات لمعالجتها، ثم تعيد القيمة الناتجة (Return Value) للمستدعي.
- وظيفتها: تمثيل الدورة الكاملة لمعالجة البيانات (مدخلات ، معالجة ، مخرجات)، وهي الأكثر مرونة واستخداماً في بناء التطبيقات.

```

Run | Debug
void main() {
  sum1();
  print(sum2());
  sum3(3, 4);
  print(sum4(3, 4));

  lambda1();
  print(lambda2());
  lambda3(5, 5);
  print(lambda4(10, 10));
}

sum1() {
  print(3 + 4);
}
lambda1() => print(5 + 5);

sum2() {
  return 3 + 4;
}
lambda2() => 3 + 4;

sum3(x, y) {
  print(x + y);
}
lambda3(x, y) => print(x + y);

sum4(x, y) {
  return x + y;
}
lambda4(x, y) => x + y;

```

**تعبير لا مبدا (Lambda Expression / Arrow Syntax):** هو أسلوب كتابة مختصر جداً يُستخدم في لغة Dart لتسهيل صياغة الدوال التي يتكون جسمها (Body) من سطر برمجي واحد فقط.

- يتم استبدال الأقواس المربعة {} وكلمة return بالرمز => (يُسمى Arrow Operator).
- **آلية العمل:** يقوم الـ Arrow Operator تلقائياً بحساب السطر الذي يليه وإرجاع قيمته (Return) للمستدعي دون الحاجة لكتابة كلمة return صراحة (كما ظهر في lam2 و lam4).

```
void main() {
  lambda1();
  lam1();
  print(lambda2());
  print(lam2());
}
```

```
// lambda expression من فوائد ال
// انه يمكن تخزينها في متغير
lambda1() => print(5 + 5);
var lam1 = () => print(5 + 5);

lambda2() => 3 + 4;
var lam2 = () => 3 + 4;
```

```
void main() {
  var list = [1, 2, 3];

  list.forEach((item) {
    print(item);
  });

  list.forEach((item) => print(item));
}
```

**الدالة المجهولة (Anonymous Function):** هي دالة تُصاغ دون أن تحمل اسماً (Nameless Function)، وتُعامل في لغة Dart كأبي كائن (Object) يمكن تمريره كعامل لدوال أخرى أو حفظه داخل متغير.

- وجه الشبه مع ال-Lambda: الدالة المجهولة غالباً ما تُكتب بأسلوب ال-Lambda (مستخدمة ال =>) عندما تتكون من سطر واحد، وذلك لزيادة مرونة واختصار الكود.

### من فوائد ال-Lambda Expression

١. انه يمكن حفظ الدوال كقيم داخل متغيرات (Functions as First-Class Citizens): في لغة Dart، تُعامل الدوال كـ "مواطنين من الدرجة الأولى"، وهذا يعني علمياً أنه يمكن معاملة الدالة تماماً كأبي قيمة أو متغير عادي (مثل الأرقام والنصوص)؛ فيمكن تخزينها داخل متغير، وتمريرها كعامل لدوال أخرى.

- حيث قمنا بإنشاء دالة مجهولة الاسم (Anonymous Function) تمثلها الأقواس ()، ثم قمنا بحفظ هذه الدالة بالكامل داخل متغير اسمه lam1. الاستدعاء: يتم استدعاؤها عبر المتغير كأنه اسم الدالة تماماً: lam1()

٢. مرونة التصميم: تتيح لنا تمرير كتل برمجية كاملة (دوال) كمعاملات داخل عناصر واجهات Flutter (مثل تمرير دالة مجهولة لحدث الضغط على زر onPressed: () => print('Button Clicked')).

٣. اختصار كود التطبيق: بدلاً من كتابة دالة كاملة مستقلة خارج نطاق العمل من أجل سطر برمجي واحد، نقوم بصياغتها وحفظها محلياً في مكان استخدامها مباشرة.

```
// 1
void sum(x, y) {
  print("sum=${x + y}");
}

// 2
var d = (x, y) => print("sum=${x + y}");
Run | Debug
void main() {
  higherOrder(4, 8, sum); // d
  //(x, y) => print("sum=${x + y}");
}

//3
higherOrder(int a,int b,Function c) {
  c(a, b);
}
```

```
void main() {
  higherOrder(4, 8, (x, y)=>print("sum=${x+y}"));
}

higherOrder(int a, int b, Function c) {
  c(a, b);
}
```

**مفهوم Higher-Order Functions:** موضوع الـ (الدوال من الرتبة العليا) هو أحد أقوى مفاهيم البرمجة الوظيفية (Functional Programming)، وهو العمود الفقري الذي بُنيت عليه أطر عمل حديثة مثل Flutter.

علمياً، تُسمى الدالة Higher-Order Function إذا كانت تحقق واحداً على الأقل من الشرطين التاليين:

- تستقبل دالة أخرى كمعامل (Parameter).
- أو تُرجع دالة أخرى كناتج (Return).

### من فوائد الـ Higher-Order Function

- عادة استخدام الكود (Reusability): يمكننا من كتابة دالة ذات هيكل عام، وتمرير السلوك أو العملية البرمجية إليها ديناميكياً من الخارج.
- الفصل بين الهيكل والتنفيذ (Abstraction): نحدد "متى وأين" يتم الاستدعاء، ونترك "كيفية التنفيذ" للدالة الممررة. (هذا هو المبدأ الأساسي لأزرار Flutter مثل onPressed التي تستقبل دالة لتنفيذها عند الضغط).

## أنواع طرق تمرير المعاملات في Dart

في الدالة `information_1`: طريقة المعاملات الاختيارية المعتمدة على الموقع (Optional Positional Parameters).

- تم التمرير من خلال : الأقواس المربعة [ ].
- المعاملات الأولى (id, name) إجبارية.
- المعاملات داخل الأقواس المربعة [agg, phone, city] اختيارية، وإذا تم تمريرها يجب أن تلتزم بالترتيب والموقع بدقة (مثلاً: لا يمكن تمرير phone بدون تمرير agg أولاً).

في الدالة `information_2`: طريقة المعاملات الاختيارية المُسمّاة (Optional Named Parameters).

- تم التمرير من خلال: الأقواس المجعدة { }.
- المعاملات داخل الأقواس {name, agg, phone, city} اختيارية ومُسمّاة؛ أي عند استدعائها يجب كتابة اسم المعامل تلو قيمته (مثل (name: "abdulsalam").
- ميزتها: لا تلتزم بالترتيب أبداً، ويمكن تمرير أي معامل منها وتجاهل البقية بحرية تامة.

في الدالة `information_3`: طريقة المعاملات المُسمّاة ذات القيم الافتراضية (Named Parameters with Default Values).

- تم التمرير من خلال: علامة اليساوي = داخل الأقواس المجعدة { }.
- تجمع هذه الطريقة بين ميزة المعاملات المسماة وإسناد قيم افتراضية (Default Values) لها (مثل (city = "Taiz").
- آلية العمل: إذا قام المستدعي بتمرير قيمة للمتغير فسيتم استخدام القيمة الممررة، أما إذا لم يمررها (مثل استدعاء (information\_3(3, "alshossi") فسيقوم البرنامج تلقائياً باستخدام القيم الافتراضية المحددة مسبقاً بدلاً من إرجاع قيمة فارغة (null).

```
void main() {
  // طرق تمرير البيانات الافتراضية عبر الدوال
  information_1(1,"Suliman",17,777777,"Ibb");
  information_2(2,name:"abdulsalam",agg:40);
  information_3(3, "alshossi");
}
```

```
information_1(id, name, |
  [agg, phone, city]) {
  print("id=$id");
  print("name=$name");
  print("agg=$agg");
  print("phone=$phone");
  print("city=$city");
}
```

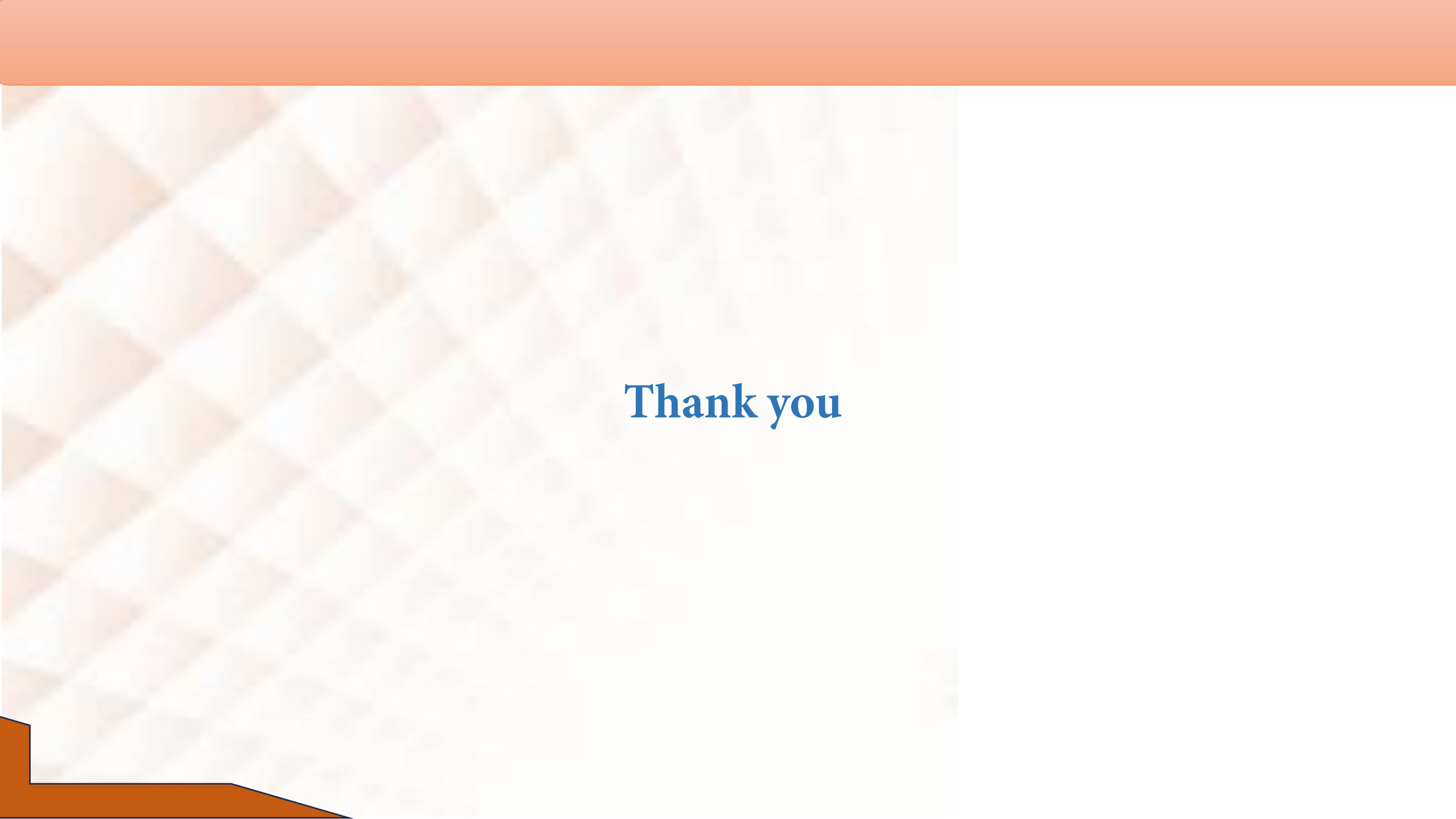
```
id=1
name=Suliman
agg=17
phone=777777
city=Ibb
```

```
information_2(id,
  {name, agg, phone, city}) {
  print("id=$id");
  print("name=$name");
  print("agg=$agg");
  print("phone=$phone");
  print("city=$city");
}
```

```
id=2
name=abdulsalam
agg=40
phone=null
city=null
```

```
information_3(id,name,{agg=45,
  phone=888888,city="Taiz"}) {
  print("id=$id");
  print("name=$name");
  print("agg=$agg");
  print("phone=$phone");
  print("city=$city");
}
```

```
id=3
name=alshossi
agg=45
phone=888888
city=Taiz
```



**Thank you**